10. nc/ncat and socat: Swiss Army Networking

10. nc/ncat and socat: Swiss Army Networking

Phase: Phase 3 — Host networking toolkit
Estimated time: 1–2 days
Slug: `10-nc-socat`

Learning objectives

- Test TCP/UDP ports with nc
- Create quick lab listeners
- Banner-grab carefully in labs
- Know socat for advanced relays

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

You need a quick TCP check, a one-shot HTTP listener, or a port forward without standing up nginx. `nc`/`socat` are the multitool — dangerous in the wrong hands, indispensable in a lab.

Core concepts

What you must be able to explain:

- Test TCP/UDP ports with nc
- Create quick lab listeners
- Banner-grab carefully in labs
- Know socat for advanced relays

Mental model for `10-nc-socat` (exercise kind `nc_vz`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
ip -br addr; ip route; ip route get 1.1.1.1
```

Sample output:

```
eth0 UP 192.168.1.20/24
default via 192.168.1.1 dev eth0
1.1.1.1 via 192.168.1.1 dev eth0 src 192.168.1.20
```

Interpretation: `ip route get` shows the chosen route, egress device, and preferred source — gold for 'why did it leave this NIC?' questions.

Example 2

```
ping -c 3 1.1.1.1
ss -tulpn | head
# macOS: netstat -anv | head
```

Sample output:

```
3 packets transmitted, 3 received
udp ... :53
tcp LISTEN ... :22
```

Interpretation: ICMP OK proves basic L3; ss proves local listeners. Combine with ARP/neigh when LAN peers fail.

Example 3

```
ip neigh show | head
ping -c 1 $(ip route | awk '/default/ {print $3; exit}')
```

Sample output:

```
192.168.1.1 dev eth0 lladdr ... REACHABLE
64 bytes from gateway
```

Interpretation: Neighbor/ARP state validates L2 adjacency to the gateway before you blame the wider internet.

Mini-lab

1. Re-type every worked command for **nc/ncat and socat: Swiss Army Networking** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.

4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
Leaving lab listeners on 0.0.0.0 in u…	See common mistakes	Re-run lesson commands
Assuming UDP nc success means healthy…	See common mistakes	Re-run lesson commands
Using nc for production TLS (prefer o…	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for nc/ncat and socat: Swiss Army Networking

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Leaving lab listeners on 0.0.0.0 in untrusted networks
- Assuming UDP nc success means healthy service
- Using nc for production TLS (prefer openssl/curl)

Exercises

Work in `exercises/10-nc-socat/`. Read that folder's `README.md` before coding.

```
cd exercises/10-nc-socat
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `nc_port_check_flags`, `nc_listen_flag`, `swiss_army_relay`. Each function is a tiny retrieval drill for **nc/ncat and socat: Swiss Army Networking**

— return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/10-nc-socat/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. nc flag for zero-I/O port scan style connect?
2. How listen on 9090?
3. Why UDP checks are weaker?

Further reading

- https://nmap.org/ncat/